JavaScript Functions Practice

Writing and Combining Functions — Practice Worksheet

Name: _____ Section: _____ Date: _____

How to use this sheet: Most of your time goes into **writing and running code**, not discussion. Work through the problems in order; they get progressively harder. Test every function in your console before moving on, and compare your result with the **Check** line under each question.

Part 1 — Basic Function Practice

Q1. Write a function `cubeNumber(n)` that **returns** the cube of `n` (n × n × n).

> **Check:** `cubeNumber(3)` → `27`

Q2. Write a function `isAdult(age)` that **returns** `true` if `age` is 18 or above, and `false` otherwise. Then rewrite it in a **single line** as `return age >= 18;` — why does that work?

> **Check:** `isAdult(18)` → `true`, `isAdult(17)` → `false`

Concept — Local Variables (Scope). A variable declared with `let` or `const` inside a function is **local**: it is created when the function runs and exists **only inside that function**. Code outside the function cannot see it.

Q3. Write a function `calculateBMI(weight, height)` that uses a **local variable** `bmi` to compute `weight / (height * height)`, and returns it. After calling it, run `console.log(bmi)` **outside** the function — what happens, and why?

> **Check:** `calculateBMI(70, 1.75)` → `22.857142857142858`

Q4. Write a function `gradeFromMarks(marks)` that returns `"A"` for 90 and above, `"B"` for 75–89, `"C"` for 50–74, and `"F"` below 50 — using **multiple `return` statements**.

> **Check:** test the boundaries 90, 89, 75, 74, 50, 49 → `"A"`, `"B"`, `"B"`, `"C"`, `"C"`, `"F"`

Q5. Write a function `applyDiscount(price, isMember = false)` that returns `price` reduced by 10% if `isMember` is `true`, and the full `price` otherwise — using a **default parameter** for `isMember`.

> **Check:** `applyDiscount(500)` → `500`, `applyDiscount(500, true)` → `450`

Q6. Write a function `countVowels(word)` that **loops** through each character of `word` and returns how many of them are vowels (a, e, i, o, u — lowercase only).

> **Hint:** `word.length` gives the number of characters and `word[i]` gives the character at position `i`, so loop `i` from `0` to `word.length - 1`.

> **Check:** `countVowels("banana")` → `3`

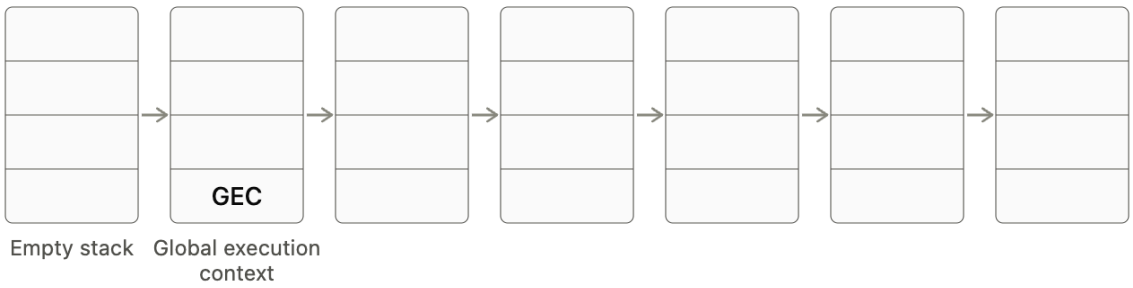
Part 2 — Functions Inside Functions

Q7. Write two small functions, `square(n)` and `cube(n)`. Then write a third function, `sumOfSquareAndCube(n)`, that **calls both of them inside its own body** and returns their sum.

> **Check:** `sumOfSquareAndCube(3)` → `36`

Concept — Global Execution Context & the Call Stack. When a script starts running, JavaScript first creates the **Global Execution Context (GEC)**, which is the environment that holds all your top-level variables and functions. It goes at the **bottom of the call stack**. Each time a function is **called**, a new execution context for it is **pushed** on top of the stack. When that function **returns**, its context is **popped** off, and control goes back to the one below it.

Q8. Trace through `sumOfSquareAndCube(3)` step by step: which function gets called first, what value does it return, and how does that lead to the final answer? Fill **one box for each change to the stack** (every push or pop) — your last box should show the stack right after `cube` returns. Then say in your own words what happens to the stack after that.



Q9. Write two functions: `isPositive(n)`, which returns `true`/`false` (treat 0 as positive, i.e. `n >= 0`), and `absoluteValue(n)`, which returns `n` made positive. Then write `safeSquareRoot(n)` that: calls `isPositive(n)` — if `true`, returns `Math.sqrt(n)` directly; if `false`, first calls `absoluteValue(n)` and returns the square root of *that* instead.

> **Hint:** `Math.sqrt(16)` gives `4`.

> **Check:** `safeSquareRoot(16)` → `4`, `safeSquareRoot(-16)` → `4`

Part 3 — Functions + Arrays & Objects

Q10. Write a function `findMax(numbers)` that returns the **largest number** in an array — using a **loop** (not `Math.max`).

> **Check:** `findMax([4, 19, 7, 12])` → `19`

Q11. Using the array below, write a function `totalPrice(products)` that **loops** over the array and returns the **sum of all the `price` values**.

```
const products = [
  { title: "Pen", price: 10 },
  { title: "Notebook", price: 45 },
  { title: "Bag", price: 500 }
];
```

> **Check:** `totalPrice(products)` → `555`